

Fine-Grained Computation Offload for Off-the-Shelf Servers in Tens of Lines

Bojie Li
Pine AI

Abstract

Hardware accelerators now sit on the critical path of online serving—GPUs, FPGAs, and increasingly *remote* services such as hardware security modules, post-quantum KEMs, and inference servers. For *fine-grained* offloads (microseconds to a few milliseconds) the classic responses to the resulting stall both fail: a context switch costs as much as the offload, and a busy-wait burns the core. Overlapping the offload with other requests is the fix, and prior systems obtain it by *adding* concurrency: an async-framework rewrite, a new runtime or dataplane OS, or a hand-tuned point integration.

We observe that the concurrency already exists: serving concurrent requests *is* suspending and resuming them, so every server ships the machinery overlap needs. Overlap is then a *routing* problem, not a rewrite problem: submit the offload to an executor, suspend the request with the server’s own deferred-response primitive, resume it on completion. Across ten off-the-shelf servers spanning every production concurrency model, this recipe takes **22–138 lines added, at most one modified**, and recovers 1.2–5.4 $\times$ on real hardware; the server’s concurrency model and the offload’s weight predict both numbers in advance, and the win is bounded by device throughput and the server’s own overlap capacity. At the limit, an LD_PRELOAD fiber runtime injects the reroute into an unmodified thread-per-connection binary (17.3 $\times$) within a characterized envelope. Rerouting suspends run-to-completion atomicity; a measured taxonomy confines the hazard to unlocked shared aggregates, and a transparent page-protection detector guards exactly those, validated on stock Redis.

Code: <https://github.com/19PINE-AI/transparent-offload>

Website: <https://01.me/research/transparent-offload>

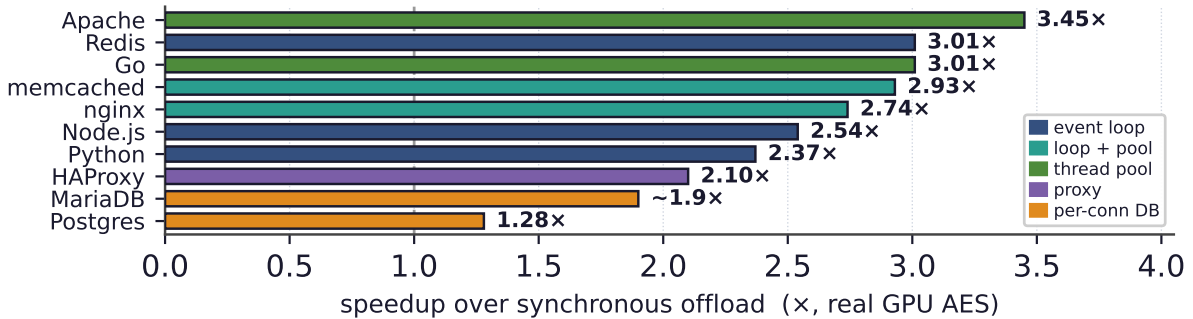
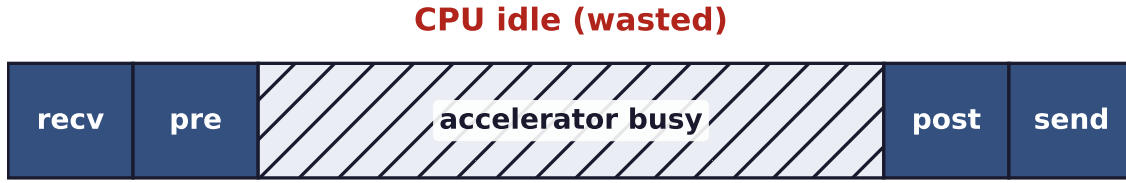


Figure 1. The headline result. Rerouting the offload through each server’s own concurrency recovers 1.3–3.5 $\times$ across ten servers (real GPU AES, 1 MiB; up to 5.4 $\times$ at 8 MiB; the databases pipeline a launch-bound op); zero-edit rerouting from outside the binary reaches 17.3 $\times$ (§4). Color marks the concurrency model.

Synchronous offload: one request, CPU stalls



Overlapped offload: CPU serves B, C while A's offload is in flight



Figure 2. The problem. A serving request is *receive* → *pre-process* → *offload* → *post-process* → *send*. Top: with a synchronous offload, the CPU stalls while the accelerator works. For fine-grained offloads (microseconds to a few milliseconds), a context switch to fill the gap costs as much as the gap itself, and busy-waiting wastes the core. Bottom: *overlap* fills the gap with other requests’ processing.

1 Introduction

Hardware accelerators are now a standard part of online serving. GPUs, TPUs, and FPGAs accelerate inference, image processing, encryption, and compression, and a growing share of “acceleration” is in fact a *remote* call: to a hardware security module, a post-quantum key-encapsulation service [National Institute of Standards and Technology, 2024], or a co-located inference server [Crankshaw et al., 2017]. Across these cases the application has the same shape: receive a request, pre-process, invoke an intensive routine, post-process, respond. On an accelerator that routine becomes an *offload*: a submission to the device and a wait for its completion, lasting microseconds to a few milliseconds. This paper is about a deceptively simple question: *what should the CPU do while it waits?*

The textbook answers fail exactly in this regime (Figure 2). *Blocking* lets the operating system run another thread, but a context switch costs several microseconds, comparable to the offload itself; this is the “killer microsecond” [Barroso et al., 2017]: the switch wastes CPU *and* adds a wakeup delay to the request’s tail. *Busy-waiting* hides the wakeup but burns a core doing nothing. The only principled answer is to *overlap* the offload with the processing of other requests.

Overlap requires concurrency at the offload point, and the systems community has treated that concurrency as something to be *added*: write the application against an asynchronous framework from the start [Provos and Mathewson, 2026, ScyllaDB, 2026], adopt a new runtime or dataplane operating system [Belay et al., 2014, Ousterhout et al., 2019, Fried et al., 2020], or hand-integrate one offload into one server [Jang et al., 2011, Intel Corporation, 2026]. All of these assume the server lacks what overlap needs.

Our starting observation is that **it already has it**. Serving concurrent connections *is* sus-

pending and resuming requests: an event loop parks a connection whenever it waits for a socket; a worker pool parks whole requests in its queue; a goroutine scheduler parks tasks at every blocking call; a database parks each connection in its own backend. Every modern server therefore ships, in production quality, exactly the machinery that overlap requires. Hiding a fine-grained offload is not a rewrite problem but a **routing problem**: at the offload call site, *submit* the work to a background executor, *suspend* the request using the server’s own deferred-response primitive, and *resume* it to reply on completion.

We validate this recipe on **ten** off-the-shelf servers spanning every concurrency model in production use: Redis, Node.js, Python/asyncio, nginx, memcached, Apache, Go, HAProxy, PostgreSQL, and MariaDB. Across the ten, the change is **22–138 lines added and zero or one existing line modified**—the core of the Redis integration is a dozen lines (Listing 1)—and on real hardware it recovers $1.2\text{--}5.4\times$ across servers and offload weights, with no failed requests (Figure 1). The cost is small for a structural reason, not a lucky one: the integration only bridges the offload to machinery that already exists, so its size is the distance to the server’s nearest suspend/resume primitive.

That structure makes the outcome *predictable*. Two properties of the deployment fix both the speedup and the code cost in advance: the server’s *concurrency model* determines what a synchronous offload costs and therefore what rerouting recovers, and how many lines the reroute takes; the offload’s *weight relative to per-request CPU work* determines whether overlap pays at all. The win is bounded at both ends—by the accelerator’s throughput and by the concurrency the server itself can express—and the model locates every server we measured, including the ones where overlap does *not* help.

Two questions remain, and they complete the paper. First, *can the change be zero?* When there is no source access at all, the libc symbol boundary is itself a suspension layer: an LD_PRELOAD fiber runtime injects the rerouting from outside an unmodified binary and reaches $17.3\times$ on a thread-per-connection server—within an envelope we characterize precisely, because transparency extends exactly as far as the behavior that flows through the interposed layer. Second, *what does rerouting break?* Suspending a handler mid-request forfeits the run-to-completion atomicity that event-driven servers silently rely on—a hazard we measure, scope, and guard.

In summary, this paper makes three contributions:

- **An offload-rerouting method.** Overlapping a fine-grained offload requires rerouting it through concurrency the server already has, not adding concurrency. One three-step method, instantiated on ten off-the-shelf servers with 22–138 lines added and at most one existing line modified, recovers $1.2\text{--}5.4\times$ on real hardware across offload weights (§3, §6).
- **A predictive model of speedup and code cost.** Concurrency model and offload weight predict the win and the cost in advance (§2); at the zero-edit limit, an interposition-envelope principle and a syscall-profile classifier predict where transparent rerouting is possible at all (§4).
- **A correctness taxonomy and a transparent guard.** Rerouting breaks run-to-completion atomicity; a measured taxonomy confines the hazard to unlocked shared aggregates, and a transparent conflict detector protects exactly those with no application changes (§5).

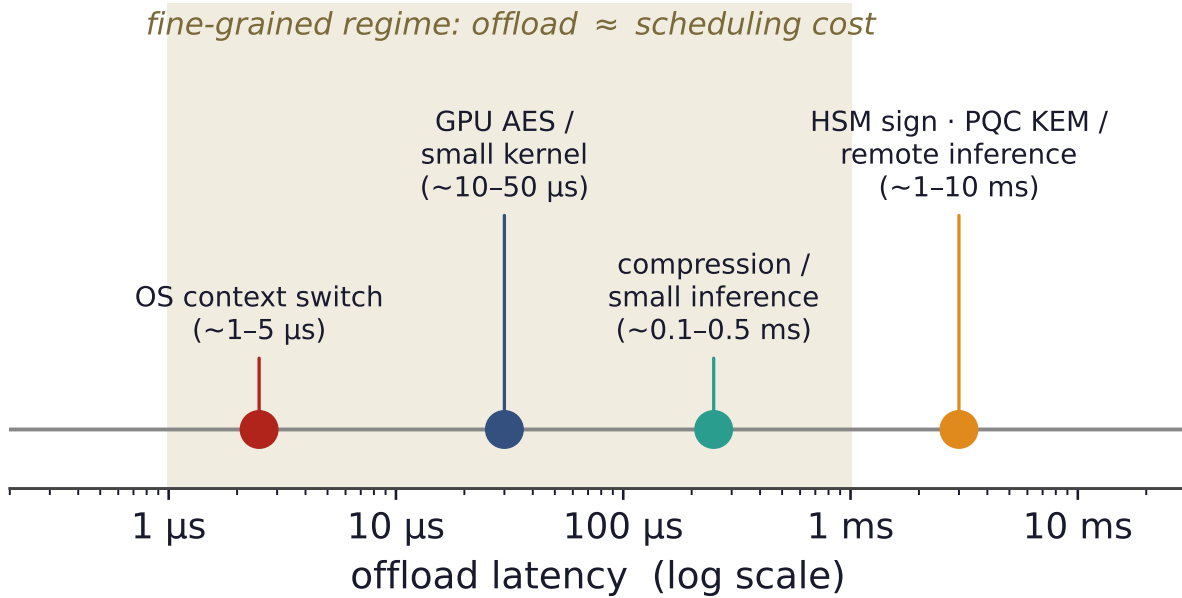


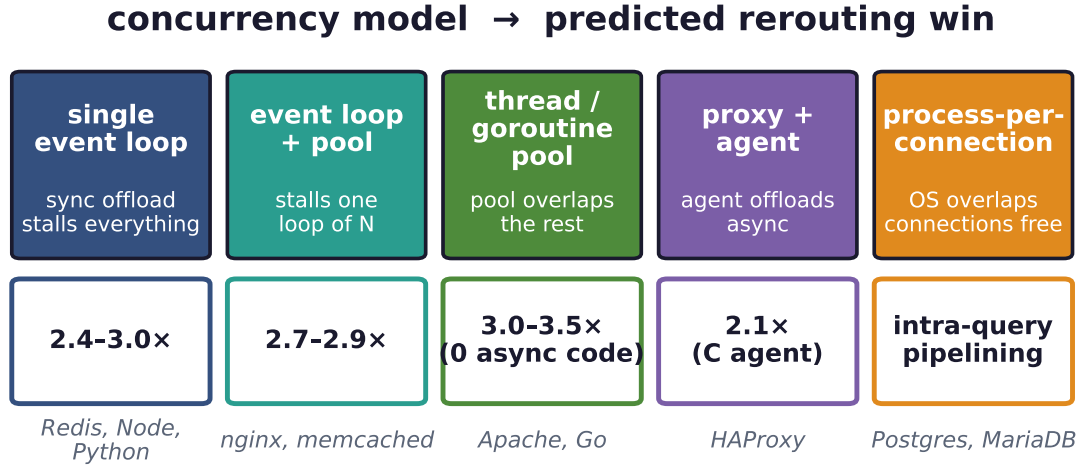
Figure 3. The fine-grained regime. Accelerator offloads span microseconds to milliseconds. In the shaded band, the offload latency is comparable to an OS context switch, so blocking pays a switch and a wakeup that rival the work itself, and busy-waiting wastes a core.

2 The Fine-Grained Regime and a Predictive Model

The fine-grained regime. An accelerator turns a CPU-bound routine into a device operation: the CPU prepares an input buffer, submits it (a kernel launch, a DMA, or a network send), and later collects the result. These round-trips span microseconds to milliseconds (Figure 3), but all are *fine-grained* relative to operating-system scheduling: the offload finishes on the timescale of a context switch, so blocking and busy-waiting are both wasteful, and overlap is the only approach that neither wastes the CPU nor inflates latency.

A taxonomy of concurrency models. The thesis says to reroute the offload through the server’s own concurrency, so the first question about any server is where that concurrency lives. Five models cover production practice. A *single-event-loop* server (Redis, Node.js, a Python `asyncio` service) multiplexes all connections on one thread; its suspension primitive is the deferred reply. An *event-loop-with-pool* server (nginx, memcached) runs a few such loops plus a worker pool. A *thread- or goroutine-pool* server (Apache, Go) parks whole requests in pooled workers or runtime-scheduled tasks. A *proxy* (HAProxy) can route requests through a native offload engine to an external agent. A *process- or thread-per-connection* server (PostgreSQL, MariaDB) gives each connection its own backend, which the OS suspends and resumes wholesale.

Prediction 1: concurrency model determines speedup and code cost. The same taxonomy predicts what a *synchronous* offload costs and therefore what rerouting recovers (Figure 4). The extreme case is the single event loop: one thread handles every connection, so a synchronous offload stalls *all* of them and throughput collapses to one request per offload latency while the hardware idles—a pathology of *structure*, not capacity—and the few-line reroute is dramatic there. In an event loop with a pool, a synchronous offload stalls only one loop of several. A



Win grows with offload weight; it pays only when the offload outweighs per-request CPU.

Figure 4. Prediction 1: the concurrency model determines what a synchronous offload costs and therefore how much rerouting recovers, from *dramatic* (a blocked single loop) to *automatic* (a pool that overlaps for free) to *intra-query* (databases that already overlap across connections). Example servers are shown under each regime.

thread or goroutine pool overlaps offloads *automatically*: the reroute needs no asynchronous code at all, because parking the worker *is* the routing. For a proxy, the reroute is configuration plus an external agent. And a per-connection database already overlaps *across* connections for free (the OS runs the backends concurrently), leaving *intra-query* pipelining of a serial offload loop as the only win.

The *code cost* follows the same axis: where the server exposes a plugin API or a deferred-response primitive the reroute is purely additive; where it exposes none (memcached), the developer must add the suspend/resume transition to the request state machine by hand, which is where the line count comes from—including the only modified line.

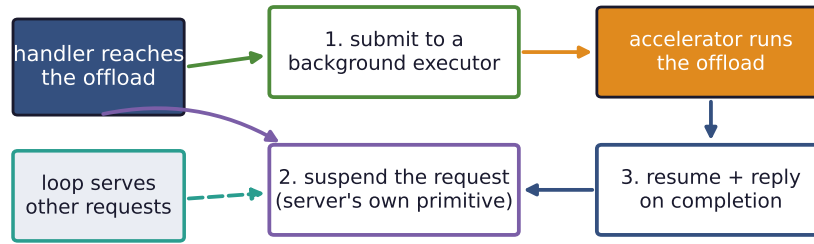
Prediction 2: offload weight determines whether overlap pays. The second property is independent of the server. Overlap reclaims the CPU time the offload would have wasted, so if per-request CPU work already rivals the offload there is little to reclaim; the win grows with the offload’s weight until a ceiling. There are two ceilings: the *accelerator’s throughput* (overlap fills the device but cannot exceed it) and the *server’s own overlap capacity* (the concurrency its machinery can keep in flight). Real speedups live between these bounds. Section 6 confirms both predictions, including a block-size sweep that traces the weight dependence on real hardware (Figure 11) and the servers where overlap correctly buys nothing.

Together the two properties form a map: given a server’s concurrency model and an offload’s latency, an operator can predict before writing any code whether to bother, how large the win will be, and roughly how many lines it will take.

3 Offload Rerouting on Ten Servers

The recipe is the thesis made concrete, and it is uniform across servers (Figure 5):

1. at the offload call site, *submit* the work to a background executor instead of waiting;

The recipe: reroute the offload through the server's existing suspend/resume machinery

22–138 lines added, at most 1 modified — the machinery already exists; one only reroutes the offload.

Figure 5. The rerouting method. Submit the offload to a background executor, suspend the request with the server’s own deferred-response primitive, and resume it to reply on completion. Meanwhile the server’s loop serves other requests. The edit is small because the suspend/resume machinery already exists; one only reroutes the offload through it.

```

1  /* accel.async: submit, suspend, resume on completion */
2  int cmd_async(RedisModuleCtx *ctx, ...) {
3      RedisModuleBlockedClient *bc =
4          RedisModule_BlockClient(ctx, reply_cb,
5                                  timeout_cb, NULL, 0); /* 2. suspend */
6      enqueue(bc);                                     /* 1. submit */
7      return REDISMODULE_OK; /* loop serves other clients */
8  }
9  void *worker(void *arg) { /* pool thread */
10     for (;;) { job_t j = dequeue();
11                accel_encrypt(j.buf, j.len); /* the offload */
12                RedisModule_UnblockClient(j.bc, NULL);
13            } /* 3. resume */
14 }

```

Listing 1. The reroute in Redis (condensed from the 83-line module `accel_module.c`). `BlockClient` is Redis’s own deferred-response primitive; the worker pool is the background executor. The command suspends before submitting, so the resume cannot race the suspension.

2. *suspend* the current request using the server’s own deferred-response primitive;
3. *resume* it and send the reply when the offload completes.

Listing 1 shows the recipe in Redis: the entire integration is an 83-line loadable module with zero edits to Redis itself, and its overlap core is the dozen lines of the listing. The synchronous variant of the same command simply calls `accel_encrypt` on the event-loop thread; the diff between the two is the reroute.

Table 1 instantiates the recipe across the landscape, and the integration point follows the concurrency model exactly as §2 predicts. Servers with a *plugin or extension API* take the reroute with zero core edits: a Redis [Redis Ltd., 2026] module, an nginx [F5/NGINX, 2026] thread-pool add-on, an Apache [Apache Software Foundation, 2026] content handler, PostgreSQL [PostgreSQL Global Development Group, 2026] and MariaDB [MariaDB Foundation, 2026] user-defined functions, and a Node.js [OpenJS Foundation, 2026] N-API add-on over libuv [libuv contributors, 2026]. Servers backed by a *language runtime*, whatever their concurrency model, need almost no new code: the goroutine scheduler overlaps a Go [The Go

server	integration point	+lines	mod	speedup
Redis 6.0	loadable module (BlockClient)	83	0	$3.01\times$
Node.js 22	N-API add-on (libuv queue)	34	0	$2.54\times$
Python 3.10	run_in_executor	22	0	$2.37\times$
nginx 1.18	add-on module (thread pool + aio)	112	0	$2.74\times$
memcached 1.6	state-machine patch	70	1	$2.93\times$
Apache 2.4	module (apxs), pooled workers	27	0	$3.45\times$
Go 1.18	plain blocking cgo call	28	0	$3.01\times$
HAProxy 2.4	SPOE + standalone C agent	138	0	$2.10\times$
PostgreSQL 14	C extension (intra-query)	42	0	$1.28\times^{\dagger}$
MariaDB 10.6	UDF (intra-query)	34	0	$\sim 1.9\times^{\dagger}$

Table 1. Offload rerouting on ten off-the-shelf servers (grouped by concurrency model: single event loop, event loop + pool, thread/goroutine pool, proxy, per-connection database). “+lines” is integration code added; “mod” is existing lines modified. Speedups are over the synchronous offload with a real GPU (1 MiB AES; [†]the databases pipeline a launch-bound op intra-query). Seven are stock server binaries; for Node.js, Python, and Go the server is an idiomatic handler on the stock runtime. Measurement setup: §6.

[Authors, 2026](#)] handler’s plain blocking cgo offload with *zero* asynchronous code, and a Python `asyncio` [[Python Software Foundation, 2026](#)] service reroutes through `run_in_executor` in one line of handler code. A *proxy* reroutes in configuration: HAProxy [[HAProxy Technologies, 2026a](#)] streams each request over its native Stream Processing Offload Engine (SPOE) [[HAProxy Technologies, 2026b](#)] to a standalone offload agent (the proxy itself changes only configuration; the 138-line C agent carries the offload).

The lone case that touches existing code is memcached [[memcached, 2026](#)], which exposes no deferred-response primitive, so we add the suspend/resume transition to its connection state machine by hand (70 lines, one modified). It is the exception that proves the structural claim: **the cost of the reroute is the distance to the server’s nearest suspend/resume primitive.**

We report *lines added* separately from *existing lines modified* because they carry different maintenance costs: added lines live in a module or extension and survive server upgrades; modified lines are the invasive part, and zero is achievable wherever an extension interface exists—nine of our ten servers. Figure 6 plots the whole study on one canvas: code cost against measured win, with the zero-edit limit of §4 at the origin.

4 The Zero-Edit Limit: Rerouting by Interposition

Can the reroute cost *zero* lines? When there is no source access at all—a stock binary, a vendor blob—the recipe cannot be applied from inside. But there is one suspension layer every dynamically linked binary passes through: the libc symbol boundary. Our transparent runtime is a shared library loaded by LD_PRELOAD that interposes the standard threading and I/O symbols. When the application creates a connection-handling thread, the runtime instead creates a *fiber*—a user-level thread with a register-only context switch of tens of nanoseconds [[Li et al., 2023](#)][—]and multiplexes all fibers on one *carrier* OS thread (occupying a single core). Whenever a fiber would block, on socket I/O or on the offload, the runtime switches to another runnable fiber; a scheduler polls I/O readiness and offload completions and resumes the corresponding fiber, saving and restoring per-fiber libc state such as `errno` across switches.

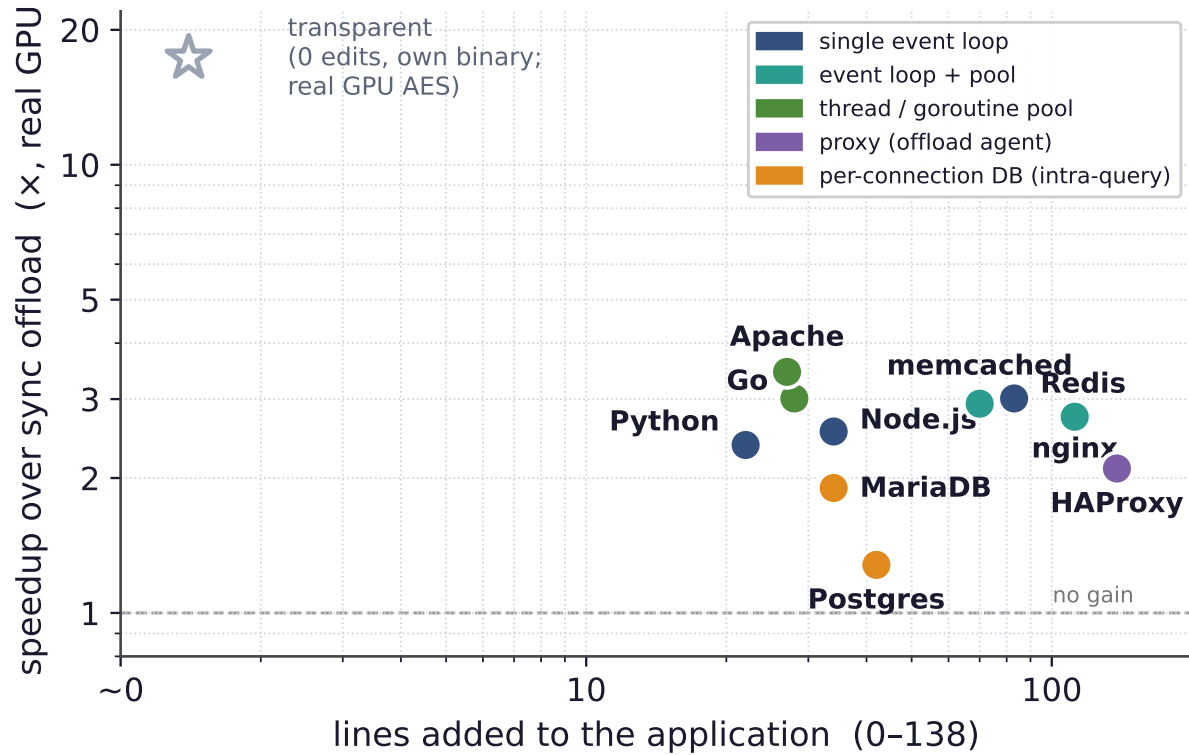


Figure 6. The study in one plot: lines of integration code vs. speedup from rerouting a real-GPU AES offload on an idle device (1 MiB blocks; the databases pipeline a launch-bound op intra-query). The few-line reroute clusters at 2–3.5 $\times$ (the offload is bandwidth-bound, so overlap fills the device but cannot exceed its throughput); the databases sit lower; the zero-edit transparent runtime reaches 17.3 $\times$ in its niche (§4). Color encodes the concurrency model.

The handler keeps its plain synchronous shape and never learns that its “thread” is a fiber: the recipe’s submit, suspend, and resume are injected from outside the binary (Figure 7).

On its home ground this works well: on a synchronous thread-per-connection handler with a real GPU performing AES, the runtime overlaps 64 connections’ offloads on one carrier thread and reaches 17.3 $\times$ the throughput of the same binary busy-waiting on each offload, with results verified bit-for-bit (11.9 $\times$ over a baseline that blocks in the driver and lets the OS overlap the 64 threads¹); a separately built DNN-inference server shows 11.8 $\times$.

Making stock binaries run under the runtime took a catalog of interposition engineering—most notably a glibc condition-variable symbol-versioning hazard that silently corrupts some binaries and that any threading-interposing tool must fix—which Appendix B records for practitioners.

The interposition envelope. The more useful contribution is the limit. A preloaded library virtualizes exactly one layer, the libc symbol boundary, so **its reach is the completeness of that layer**: behavior that escapes the layer is out of reach, and there are exactly three ways to escape it (Figure 8).

- *Below it.* A storage engine such as InnoDB synchronizes with raw `futex` system calls [Franke

¹The blocking-baseline comparison was measured while a co-tenant shared the GPU (both sides equally affected); the busy-wait comparison ran on an otherwise idle GPU.

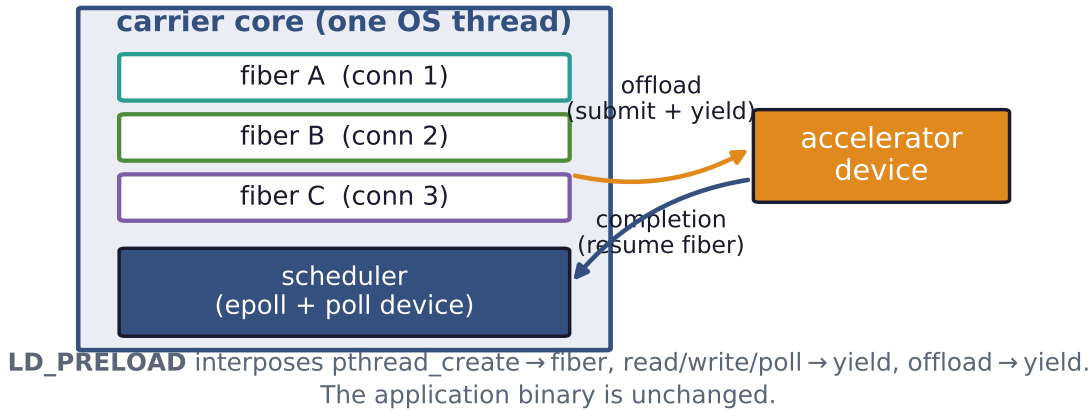


Figure 7. The zero-edit limit. An LD_PRELOAD library turns each connection thread into a fiber on one carrier thread; a fiber yields at its offload (and at socket I/O) and the scheduler runs another until the offload completes. The binary is unchanged. On a synchronous thread-per-connection handler with a real GPU this overlaps 64 connections’ offloads for a $17.3\times$ gain over a busy-wait synchronous baseline ($11.9\times$ over OS-thread blocking).

et al., 2002] issued directly, bypassing pthread; a fiber that blocks in a raw futex stalls the whole carrier, and under contention the server deadlocks—we confirmed this with a live backtrace of the frozen carrier inside InnoDB.

- *Beside it.* A managed runtime such as the JVM keeps “the current thread” in a native thread-local slot read inline from a CPU register, which symbol interposition cannot virtualize per fiber; the moment two fiberized JVM requests interleave, the runtime reads the wrong thread object and segfaults (the same mechanism applies to Go and .NET).
- *Behind it.* Overlap needs idle CPU to reclaim, and a thread-per-connection TLS terminator that spends real CPU on per-request crypto has none—the OS already overlaps its offloads across threads, while the single carrier serializes the crypto and pays an interposition tax on every yield, ending up $2\text{--}3\times$ slower than native.

These are properties of where an application places its behavior, not gaps in engineering; no interposition removes them.

A syscall-profile classifier. The envelope is predictable from the outside: we profile the per-request blocking syscalls a server makes under load (Figure 9). Event-driven servers show an `epoll`-dominated profile and have no per-connection thread to fiberize: the runtime loads safely but never engages. Thread-per-connection servers that block at the libc layer show near-zero per-request futex traffic and are fiberizable. Engines that synchronize below libc are unmistakable: InnoDB issues $\sim 13,000$ futex calls per request, some $500\times$ a fiberizable server’s, alongside asynchronous kernel I/O (`io_uring` [Axboe, 2019]). A cheap `strace` therefore tells an operator in advance whether zero-edit rerouting can work.

The verdict frames the division of labor. Zero-edit rerouting serves a real but narrow niche: thread-per-connection servers with light per-request CPU and libc-level blocking, reached with no source access. By construction it cannot help event-driven servers—the class where a synchronous offload is most catastrophic. For everyone else, the few-line recipe of §3 is the answer.

Where transparent interposition reaches — and the three walls

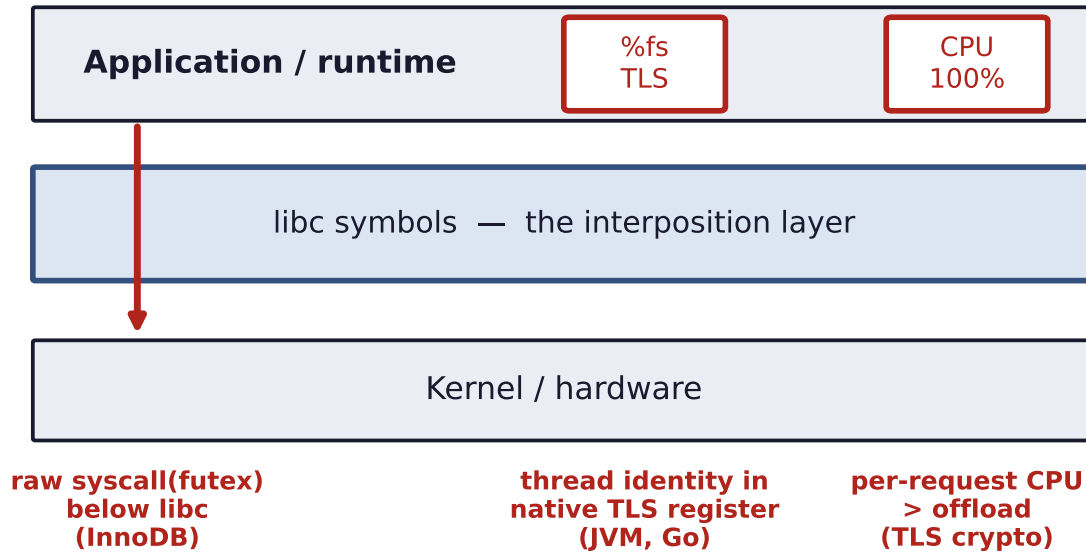


Figure 8. The interposition envelope. A preloaded library virtualizes the libc symbol layer, and reaches exactly the behavior that flows through it. Behavior escapes in three ways: below the layer (raw futex syscalls, InnoDB), beside it (thread identity in a native TLS register, the JVM and Go), and behind it (per-request CPU that already exceeds the offload).

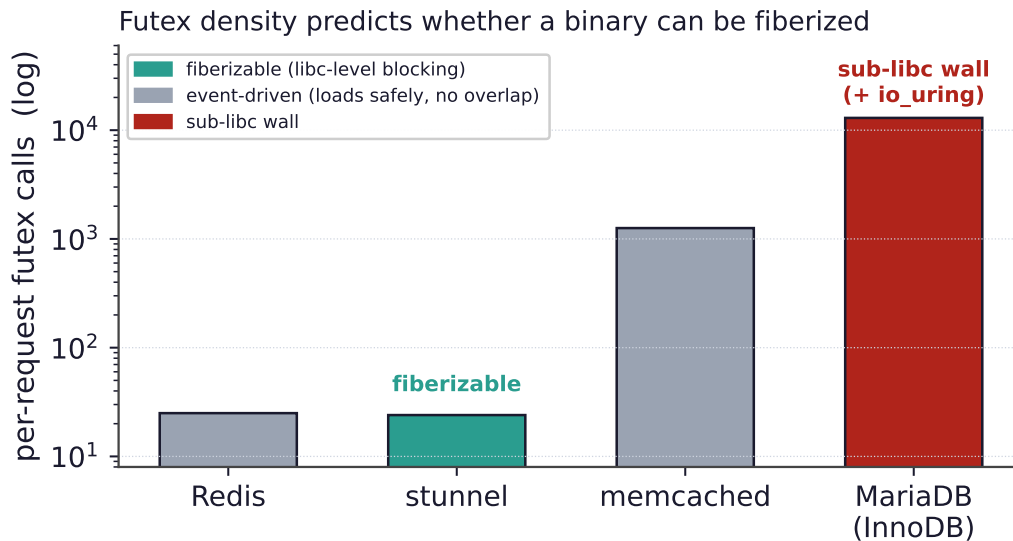


Figure 9. Predicting the envelope. Per-request futex calls separate thread-per-connection servers that block at the libc layer (fiberizable: stunnel) from engines that synchronize below it (InnoDB, $\sim 13,000$, some $500\times$ higher, plus `io_uring`). Event-driven servers (gray) have no per-connection thread to fiberize; their `epoll` profile flags them “loads safely, no overlap.”

5 Correctness under Rerouting

Rerouting has one cost that is easy to miss. A single-threaded server runs each handler to completion, so handlers observe shared state atomically—an invariant the code *silently* relies on precisely because the architecture provides it for free. Suspending a handler at its offload

breaks that invariant: if post-processing does a read-modify-write on shared state—a counter, a cache entry, a key in a store—two overlapped requests can interleave their sequences and lose updates. On a *stock Redis server* whose module command reads a key, runs a real GPU offload, and writes the incremented value back, unprotected overlap silently loses tens of thousands of updates (Figure 10, left). The more latency overlap hides, the wider the race it opens.

A measured taxonomy of shared-state patterns. To scope the problem we enumerated the shared-state patterns that real offload-adjacent servers keep, and measured each under overlapped execution in race-instrumented harnesses spanning crypto (OpenSSL) and compression (zlib). Three patterns are unconditionally safe: *read-only* shared state (model weights, dictionaries, lookup tables), *per-connection* state (which one handler per connection serializes for free), and *lock-protected* state (locks serialize regardless of scheduling). All measured zero conflicts. The only hazardous pattern is the fourth: *unlocked shared mutable aggregates*—counters, batch queues, caches—where unlocked read-modify-writes lose updates essentially every time they collide. And that pattern is exactly the state a single-threaded server keeps unlocked *because* it trusts run-to-completion atomicity—the very atomicity rerouting removes. The dividing line is the state pattern, not the application domain: bulk crypto, compression, hashing, and stateless inference land safe because the heavy routine is pure and their shared state is read-only or already locked.

A transparent conflict detector. The safe patterns need nothing. For the residual—unlocked shared aggregates—a transparent *conflict detector* restores correctness with no application change: it write-protects the shared-state pages, snapshots a version clock when a handler parks at its offload, and treats a write fault on a page modified during the park as a conflict; under enforcement it serializes only the conflicting handlers. On the stock-Redis workload it drives lost updates to *zero* while running $1.8\times$ faster than a coarse lock, which serializes the very offloads it protects; its overhead versus unprotected overlap is within measurement noise (24.7K vs. 24.1K req/s) (Figure 10, right). The detector is a property of the overlap mechanism, not of any one integration, so it applies at every point of the spectrum, including the zero-edit limit.

6 Evaluation

This section validates the two predictions of §2 on real hardware—Prediction 1 across the ten-server landscape, Prediction 2 by sweeping the offload’s weight—and closes with latency under load.

Experimental setup. All cross-server offloads are real, on real hardware, with no emulated latencies; the one exception is the open-loop latency study of Figure 12, which uses a controlled emulated offload and is labeled as such. Experiments run on a server with an NVIDIA RTX PRO 6000 (Blackwell) GPU. The GPU path performs AES [OpenSSL Project, 2026] on its own CUDA stream, polled by `cudaEventQuery` [NVIDIA Corporation, 2026], with the block size sweeping from 4 KiB (launch-bound) to 8 MiB (bandwidth-bound). GPU AES is a stand-in for offloads that beat the host CPU (a modern core’s AES-NI rivals a GPU on this cipher); we use it because it gives a cleanly tunable offload weight on real hardware. For the latency-bound

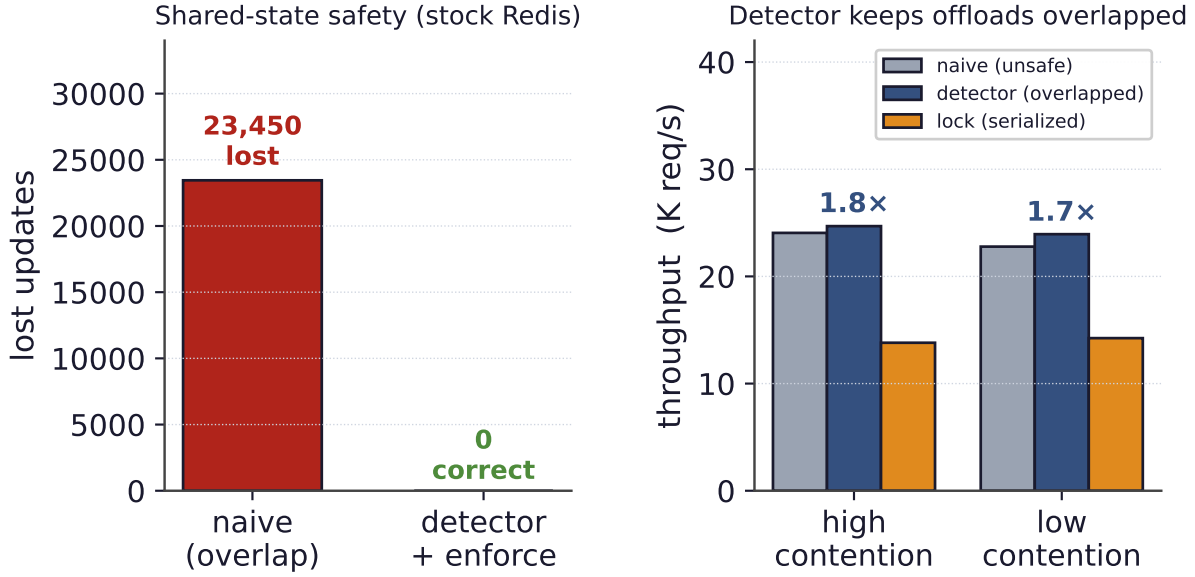


Figure 10. Guarding the one hazardous pattern, measured on a stock Redis server with a real GPU offload. Left: unprotected overlap of a shared read-modify-write loses 23,450 updates. The page-protection detector flags 26,185 conflicts and, under enforcement, reduces lost updates to zero. Right: a coarse lock serializes the very offloads it protects (13.8K req/s), while the detector keeps non-conflicting offloads overlapped (24.7K req/s), 1.8 $\times$ faster (1.7 $\times$ at low contention) and within noise of unprotected overlap (24.1K req/s).

remote class (HSM signatures, post-quantum KEMs, remote inference) we stand up a real TCP signer doing a genuine RSA-2048 signature per request. Servers are driven by standard load generators (redis-benchmark, ab).

Validation of Prediction 1 across the server landscape. On a 1 MiB GPU AES offload (realistic bulk crypto, idle GPU) the reroute recovers the win the model predicts in every regime, with no failed requests (Figure 1, Table 1): single event loops and event-loop-with-pool servers cluster around 2.4–3 $\times$, thread and goroutine pools reach 3.0–3.5 $\times$ with *no* asynchronous code, and the per-connection databases show only the predicted intra-query win with a launch-bound op. The cluster tops out at 2–3.5 $\times$ because this offload is bandwidth-bound: once overlap saturates the device, its throughput is the ceiling. The proxy isolates where the win comes from: HAProxy’s gain rides entirely on the offload *agent*, so a 21-line GIL-bound Python agent gains nothing while a 138-line C pthread-pool agent recovers 2.1 $\times$ on the same offload.

Validation of Prediction 2: offload weight and the two ceilings. Sweeping the GPU AES offload by block size on a single event loop traces the weight dependence (Figure 11; full table in Appendix A): the same reroute gives essentially nothing for a light block (1.24 $\times$ at 4 KiB, 46 μ s, where the offload is on the order of the server’s own per-request work) and rises to 5.41 $\times$ at 8 MiB (2.3 ms), approaching the GPU’s bandwidth—the first ceiling, a property of the device. A *latency-bound* offload lifts that ceiling because its device is not saturated: with our real RSA-2048 remote signer (821 μ s round-trip) the same Python server reaches 3.53 $\times$ —but not the order of magnitude a deep queue could in principle give, because the bottleneck moves to the second ceiling, the server’s own overlap capacity (throughput and round-trip latency

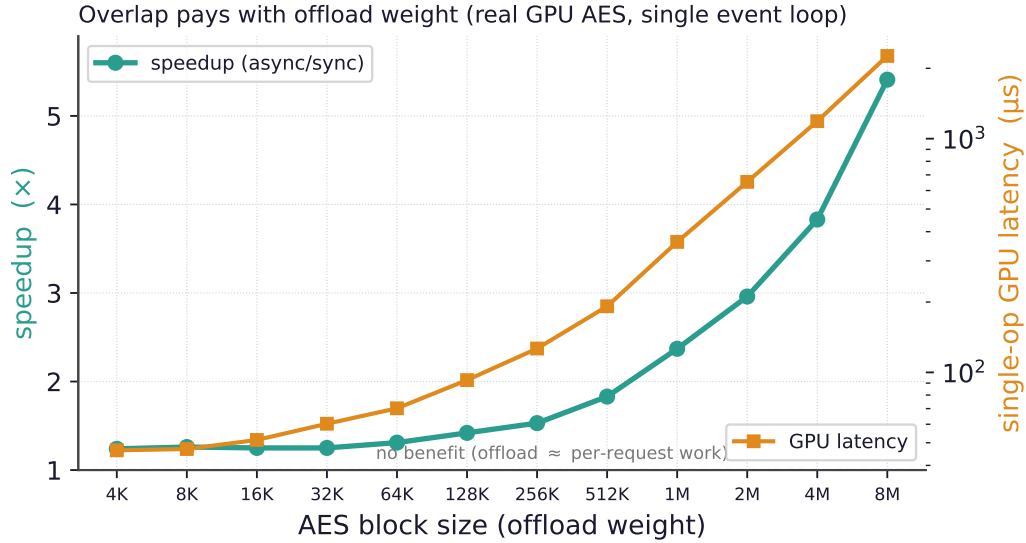


Figure 11. Prediction 2, measured (real GPU, idle): overlap pays only when the offload outweighs per-request CPU work. Sweeping a real GPU AES offload by block size (consecutive powers of two, 4 KiB–8 MiB) on a single-event-loop server, the same reroute gives little benefit for a light block ($1.24\times$ at 4 KiB, 46 μ s) and rises to $5.41\times$ at 8 MiB (2.3 ms), approaching the GPU’s bandwidth. Right axis: measured single-op GPU latency.

imply the `asyncio`/GIL executor keeps only ~ 6 requests truly in flight). Real overlap is thus bounded at both ends; across our servers and offloads it lands at $1.2\text{--}5.4\times$.

Latency under offered load. Overlap is a latency win too. By keeping the CPU busy during offloads, the overlapping path holds both median and tail (p99) latency low up to roughly *four times* the offered load that blocking can sustain before its knee (Figure 12).

7 Discussion and Limitations

When rerouting does not pay. The model doubles as a stop sign. An operator whose per-request CPU already rivals the offload has little to reclaim, and rerouting buys complexity for nothing (Prediction 2 of §2—the same condition that raises the transparent runtime’s third wall). And an operator whose accelerator is already saturated should buy device bandwidth, not rewire the server: overlap recovers wasted utilization but never manufactures throughput.

Scope of the conflict detector. The conflict detector is a targeted safety net for the one pattern rerouting endangers, not a general transactional memory; shared state outside the monitored segments, or correctness requirements beyond last-writer-wins, need stronger machinery.

8 Related Work

Figure 13 places this work among prior approaches, which either *add* the concurrency—new frameworks, runtimes, and operating systems—or hand-integrate one offload into one server; we instead reuse the concurrency existing servers already have, across the whole landscape of server architectures, and predict the cost of doing so.

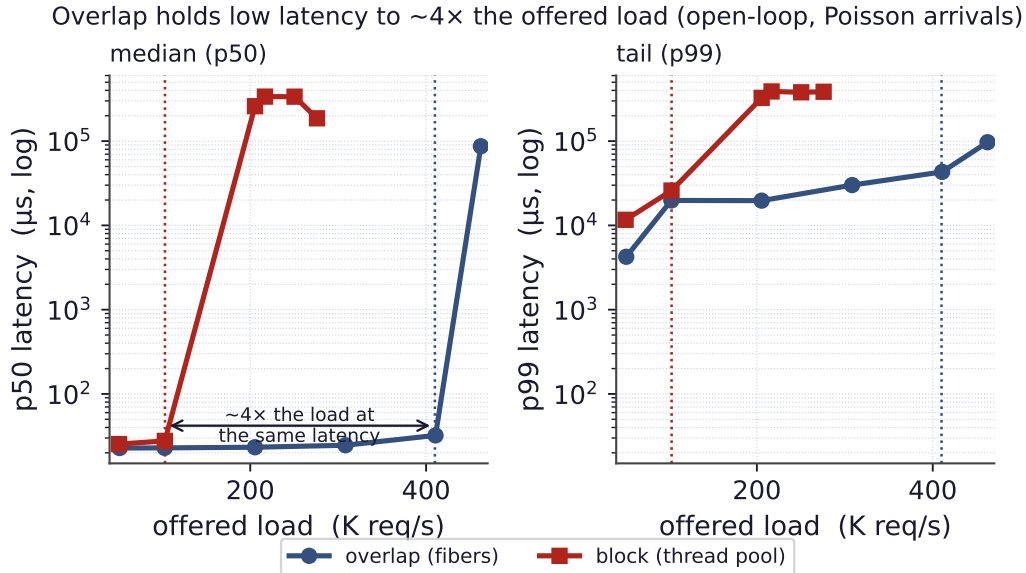


Figure 12. Latency under load (open-loop Poisson arrivals over a controlled 20μ s *emulated* offload, the one emulated experiment in the paper, used so offered load can be swept precisely; runtime/openloop.csv). Blocking on the offload drives both median (p50, left) and tail (p99, right) latency up at its saturation point near 103K req/s, while overlapping holds low latency to roughly $4\times$ that offered load (knee near 410K req/s) before its own knee.

Threads, events, and coroutines. How to hide I/O latency cheaply is an old debate between an event-driven camp that structures the server as a state machine over an event loop at the cost of “stack ripping” [Adya et al., 2002] (Flash [Pai et al., 1999], SEDA [Welsh et al., 2001]) and a user-level-thread camp that keeps the synchronous style and yields at blocking points (Capriccio [von Behren et al., 2003], State Threads [State Threads Library, 2009], libtask [Cox, 2005], core-aware Arachne [Qin et al., 2018], and language-level threads such as goroutines [The Go Authors, 2026] and Java virtual threads [OpenJDK, 2023]). Our fiber runtime is in the second lineage with a fast register-only switch [Li et al., 2023], but the contribution is not another threading library: it is to overlap an *accelerator offload* in servers built either way, quantify the modification cost across real servers, and map where the transparent form demonstrably stops.

Microsecond-scale systems. A body of work rebuilds the OS or runtime to run microsecond-scale tasks efficiently: dataplane operating systems such as IX [Belay et al., 2014], work-stealing schedulers such as ZygOS [Prekas et al., 2017], and core-reallocating runtimes such as Shenango [Ousterhout et al., 2019], Caladan [Fried et al., 2020], and Demikernel [Zhang et al., 2021], typically built over kernel-bypass or asynchronous I/O (DPDK [DPDK Project, 2026], io_uring [Axboe, 2019]). They attack the same killer microsecond problem [Barroso et al., 2017] but demand a new stack and rewritten applications. We instead ask how little it costs to overlap one offload inside existing, unmodified servers; the directions are complementary.

Full hardware offload. Another line moves the *entire* datapath onto hardware: KV-Direct [Li et al., 2017] runs a key-value store in the NIC, ClickNP [Li et al., 2016] compiles network functions to an FPGA, iPipe [Liu et al., 2019] offloads application logic onto SmartNICs (and must itself schedule the offloaded tasks’ granularity, the dual of our problem), and

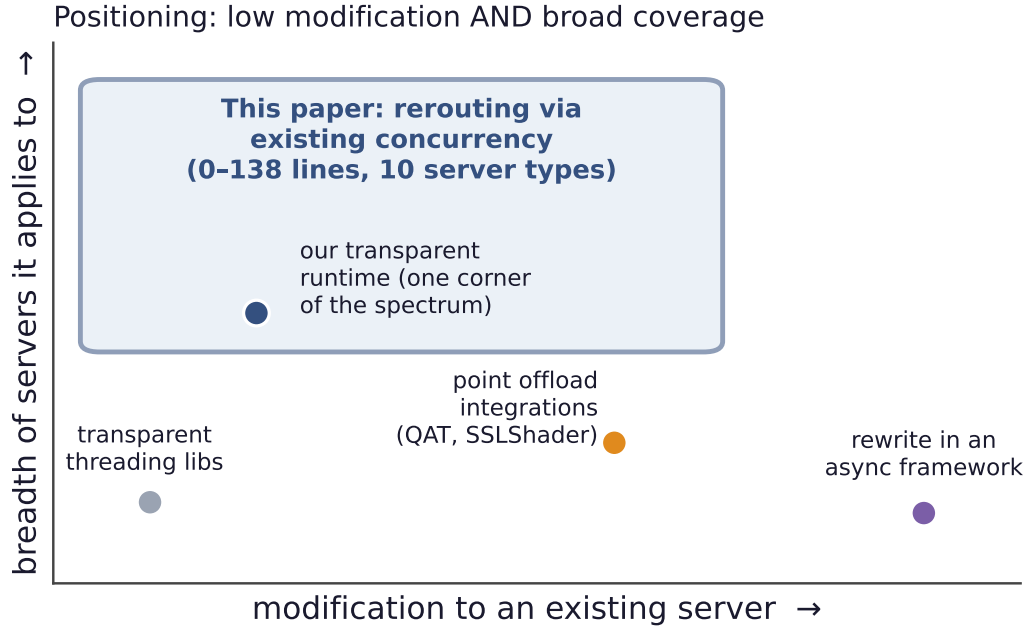


Figure 13. Positioning. Existing approaches are either low-effort but narrow (threading libraries, of which our own transparent runtime is one) or high-effort and still narrow (point integrations, async-framework rewrites). Rerouting through the server’s own concurrency reaches the desirable region: little modification to existing servers, across many server types.

GPUnet [Kim et al., 2014] lets GPU code drive the network. These eliminate the CPU’s role and require rebuilding the application around the accelerator. Our target is the opposite and far more common case: the application stays on the CPU and must hide one fine-grained step’s latency with almost no change.

Offload integrations and async frameworks. Closer to us, specific systems overlap specific offloads—TLS on GPUs (SSLShader [Jang et al., 2011]), QAT engines in nginx’s async paths [Intel Corporation, 2026], and inference servers that batch GPU work (RedisAI [RedisAI, 2022], Clipper [Crankshaw et al., 2017])—but each is a point integration tuned to one server and offload. Async frameworks (libevent [Provos and Mathewson, 2026], libuv [libuv contributors, 2026], Seastar [ScyllaDB, 2026], async/await) and proxy offload engines (HAProxy SPOE [HAProxy Technologies, 2026b], Envoy ext_proc [Envoy Project, 2026]) make overlap the default, but only if the application is written in them from the start. We instead inject overlap into existing servers and predict the win and the cost across the landscape.

Conflict detection. Dirty tracking via page protection, software distributed shared memory [Amza et al., 1996], and transactional memory [Herlihy and Moss, 1993] all detect or prevent conflicting concurrent writes. Our detector borrows the page-protection technique but is lightweight and specialized to the one hazard rerouting introduces: a read-modify-write split across an offload.

Symbol interposition. The fragility of interposing versioned glibc symbols is folklore among practitioners. Our condition-variable finding (Appendix B) documents a concrete, reproducible

instance and its fix.

9 Conclusion

What should the CPU do while it waits for the accelerator? It should overlap the offload with other requests—and it does not need a new framework, runtime, or operating system to do so, because every server that serves concurrent requests already contains the machinery overlap requires. Rerouting the offload through that machinery takes tens of lines in off-the-shelf servers across every concurrency model ($1.2\text{--}5.4\times$ on real hardware), is predictable in advance from that model and the offload’s weight, can occasionally be done with zero lines from outside the binary ($17.3\times$, within a precisely characterized envelope), and stays correct with one targeted guard for the run-to-completion atomicity it suspends. The result is a practical recipe, and a map, for hiding fine-grained accelerator-offload latency in the servers that run online services today.

Acknowledgements

This paper began as a draft written during the author’s internship at Microsoft Research in 2017. Nine years later, with the help of Pine Copilot and Claude Code, the author finally brought it to completion. The work was produced using Pine Copilot’s voice-directed *whisper coding* workflow [Pine AI, 2025], in which the author specifies, discusses, and reviews the work by voice while a coding agent—Claude Code with Claude Opus 4.8 and Claude Fable 5—carries out the planning, coding, experiments, and paper writing. The author thanks BSQL Networking for hosting the NVIDIA RTX PRO 6000 GPU.

A AES Block-Size Sweep (detailed values)

Table 2 gives the full per-size measurements behind the offload-weight curve of Figure 11, all on a real GPU on an *idle* device (no co-tenant compute), driving a single-event-loop server (Python `asyncio` with a 32-thread executor) at 50 concurrent clients. The AES block size is swept over consecutive powers of two from 4 KiB to 8 MiB. For each we report the measured single-op GPU latency (one offload in flight), the synchronous and asynchronous throughput, and their ratio. The speedup grows from $1.24\times$ at 4 KiB (launch-bound: the offload is tens of microseconds, on the order of the server’s own per-request work, so there is little to overlap) to $5.41\times$ at 8 MiB (the offload is bandwidth-bound at ~ 2.3 ms, so overlapping it across requests reclaims most of the wait), up to the GPU’s throughput ceiling.

B Interposition Obstacles for the Transparent Runtime

Running stock binaries under the `LD_PRELOAD` fiber runtime of §4 surfaced five obstacles below the architectural walls. Each was resolved once and is recorded here because any threading-interposing tool will meet them.

The condition-variable versioning hazard. The sharpest obstacle is a *symbol-versioning* hazard [Drepper, 2011]: the glibc condition-variable functions carry two incompatible ABIs under

block size	GPU latency (μ s)	sync (req/s)	async (req/s)	speedup
4 KiB	46.4	3750	4667	1.24×
8 KiB	47.0	3688	4650	1.26×
16 KiB	51.4	3657	4573	1.25×
32 KiB	60.3	3501	4371	1.25×
64 KiB	70.2	3326	4355	1.31×
128 KiB	92.7	2970	4218	1.42×
256 KiB	126.5	2706	4128	1.53×
512 KiB	191.8	2454	4494	1.83×
1 MiB	361.6	1538	3644	2.37×
2 MiB	653.6	966	2858	2.96×
4 MiB	1188.1	506	1937	3.83×
8 MiB	2258.6	227	1228	5.41×

Table 2. Real-GPU AES block-size sweep on a single-event-loop server (idle GPU). Source: `transparent-runtime/apps/aes_blocksize_py_results.csv`.

one name (GLIBC_2.2.5 and GLIBC_2.3.2), so a naive interposer that defines an *unversioned* `pthread_cond_signal` breaks the linker’s version-matched relocation and silently corrupts *some* binaries while sparing others: in our sweep it crashes MariaDB at startup with a wild pointer and is tolerated by Redis, memcached, nginx, and stunnel (Figure 14). Because a transparent runtime cannot know in advance which binaries are susceptible, version-matched interposition is mandatory: export the interposed symbols at their exact version via a linker version script and resolve the real ones with `dlvsym`.

Interposing `pthread_cond_*` without version-matching

	MariaDB	Redis	memcached	nginx	stunnel
naive (unversioned)	CRASH	OK	OK	OK	OK
versioned (our fix)	OK	OK	OK	OK	OK

Figure 14. A reusable interposition hazard. The glibc condition-variable functions carry two incompatible ABIs under one name. A naive unversioned interposer breaks the version-matched relocation and silently corrupts *some* binaries (MariaDB crashes at startup) while sparing others. Version-matching the interposed symbols fixes all of them.

Four lesser obstacles.

- **Alternate I/O entry points.** A server may do its socket I/O through `recv/send/poll` rather than `read/write`, so every blocking entry point the application can reach must be

interposed to yield the fiber.

- **Real-thread identity.** `pthread_self` must return the genuine OS thread identity, because the C library's stack-bounds logic relies on it; returning a per-fiber value breaks libc internals.
- **Carrier re-establishment after fork.** A daemon that forks drops the carrier thread in the child; the carrier must be re-established before any fiber can run.
- **Priority inversion.** Pinning the carrier to a real-time priority can invert priorities against a lock holder running at normal priority, stalling the carrier.

References

- Atul Adya, Jon Howell, Marvin Theimer, William J. Bolosky, and John R. Douceur. Cooperative task management without manual stack management. In *Proceedings of the USENIX Annual Technical Conference (ATC)*, 2002.
- Cristiana Amza, Alan L. Cox, Sandhya Dwarkadas, Pete Keleher, Honghui Lu, Ramakrishnan Rajamony, Weimin Yu, and Willy Zwaenepoel. TreadMarks: Shared memory computing on networks of workstations. *IEEE Computer*, 29(2):18–28, 1996.
- Apache Software Foundation. Apache HTTP server. <https://httpd.apache.org/>, 2026.
- Jens Axboe. Efficient IO with `io_uring`. https://kernel.dk/io_uring.pdf, 2019.
- Luiz Barroso, Mike Marty, David Patterson, and Parthasarathy Ranganathan. Attack of the killer microseconds. *Communications of the ACM*, 60(4):48–54, 2017.
- Adam Belay, George Prekas, Ana Klimovic, Samuel Grossman, Christos Kozyrakis, and Edouard Bugnion. IX: A protected dataplane operating system for high throughput and low latency. In *Proceedings of the 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 49–65, 2014.
- Russ Cox. libtask: A coroutine library for C and Unix. <https://swtch.com/libtask/>, 2005.
- Daniel Crankshaw, Xin Wang, Giulio Zhou, Michael J. Franklin, Joseph E. Gonzalez, and Ion Stoica. Clipper: A low-latency online prediction serving system. In *Proceedings of the 14th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 613–627, 2017.
- DPDK Project. DPDK: Data plane development kit. <https://www.dpdk.org/>, 2026.
- Ulrich Drepper. How to write shared libraries. <https://www.akkadia.org/drepper/dsohowto.pdf>, 2011.
- Envoy Project. External processing filter (`ext_proc`). https://www.envoyproxy.io/docs/envoy/latest/configuration/http/http_filters/ext_proc_filter, 2026.
- F5/NGINX. nginx: Http and reverse proxy server. <https://nginx.org/>, 2026.
- Hubertus Franke, Rusty Russell, and Matthew Kirkwood. Fuss, futexes and furwocks: Fast userlevel locking in Linux. In *Proceedings of the Ottawa Linux Symposium (OLS)*, pages 479–495, 2002.

- Joshua Fried, Zhenyuan Ruan, Amy Ousterhout, and Adam Belay. Caladan: Mitigating interference at microsecond timescales. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 281–297, 2020.
- HAProxy Technologies. HAProxy: The reliable, high-performance TCP/HTTP load balancer. <https://www.haproxy.org/>, 2026a.
- HAProxy Technologies. Stream processing offload engine (SPOE) and protocol (SPOP). <https://www.haproxy.com/documentation/>, 2026b.
- Maurice Herlihy and J. Eliot B. Moss. Transactional memory: Architectural support for lock-free data structures. In *Proceedings of the 20th Annual International Symposium on Computer Architecture (ISCA)*, pages 289–300, 1993.
- Intel Corporation. Intel quickassist technology (Intel QAT). <https://www.intel.com/content/www/us/en/architecture-and-technology/intel-quick-assist-technology-overview.html>, 2026.
- Keon Jang, Sangjin Han, Seungyeop Han, Sue Moon, and KyoungSoo Park. SSLShader: Cheap SSL acceleration with commodity processors. In *Proceedings of the 8th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2011.
- Sangman Kim, Seonggu Huh, Xinya Zhang, Yige Hu, Amir Wated, Emmett Witchel, and Mark Silberstein. GPUnet: Networking abstractions for GPU programs. In *Proceedings of the 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 201–216, 2014.
- Bojie Li, Kun Tan, Layong (Larry) Luo, Yanqing Peng, Renqian Luo, Ningyi Xu, Yongqiang Xiong, Peng Cheng, and Enhong Chen. ClickNP: Highly flexible and high performance network processing with reconfigurable hardware. In *Proceedings of the 2016 ACM SIGCOMM Conference*, pages 1–14, 2016.
- Bojie Li, Zhenyuan Ruan, Wencong Xiao, Yuanwei Lu, Yongqiang Xiong, Andrew Putnam, Enhong Chen, and Lintao Zhang. KV-Direct: High-performance in-memory key-value store with programmable NIC. In *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP)*, pages 137–152, 2017.
- Bojie Li, Zihao Xiang, Xiaoliang Wang, Han Ruan, Jingbin Zhou, and Kun Tan. FastWake: Revisiting host network stack for interrupt-mode RDMA. In *Proceedings of the 7th Asia-Pacific Workshop on Networking (APNet)*, pages 1–7. ACM, 2023. doi: 10.1145/3600061.3600063.
- libuv contributors. libuv: Cross-platform asynchronous I/O. <https://libuv.org/>, 2026.
- Ming Liu, Tianyi Cui, Henry Schuh, Arvind Krishnamurthy, Simon Peter, and Karan Gupta. Offloading distributed applications onto SmartNICs using iPipe. In *Proceedings of the 2019 ACM SIGCOMM Conference*, pages 318–333, 2019.
- MariaDB Foundation. MariaDB server: The open source relational database. <https://mariadb.org/>, 2026.

- memcached. memcached: A distributed memory object caching system. <https://memcached.org/>, 2026.
- National Institute of Standards and Technology. Module-lattice-based key-encapsulation mechanism standard. Technical Report FIPS 203, NIST, 2024.
- NVIDIA Corporation. CUDA C++ programming guide: Asynchronous concurrent execution. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>, 2026.
- OpenJDK. JEP 444: Virtual threads (project Loom). <https://openjdk.org/jeps/444>, 2023.
- OpenJS Foundation. Node.js: A JavaScript runtime built on V8. <https://nodejs.org/>, 2026.
- OpenSSL Project. OpenSSL: Cryptography and SSL/TLS toolkit. <https://www.openssl.org/>, 2026.
- Amy Ousterhout, Joshua Fried, Jonathan Behrens, Adam Belay, and Hari Balakrishnan. Shenango: Achieving high CPU efficiency for latency-sensitive datacenter workloads. In *Proceedings of the 16th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 361–378, 2019.
- Vivek S. Pai, Peter Druschel, and Willy Zwaenepoel. Flash: An efficient and portable web server. In *Proceedings of the USENIX Annual Technical Conference (ATC)*, 1999.
- Pine AI. Pine AI: The most natural human-computer interface is your voice. Blog post, 2025. URL <https://www.19pine.ai/blog/pine-ai-the-most-natural-human-computer-interface-is-your-voice>. Accessed 2026-07-02.
- PostgreSQL Global Development Group. PostgreSQL: The world’s most advanced open source relational database. <https://www.postgresql.org/>, 2026.
- George Prekas, Marios Kogias, and Edouard Bugnion. ZygOS: Achieving low tail latency for microsecond-scale networked tasks. In *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP)*, pages 325–341, 2017.
- Niels Provos and Nick Mathewson. libevent: An event notification library. <https://libevent.org/>, 2026.
- Python Software Foundation. asyncio: Asynchronous I/O in CPython. <https://docs.python.org/3/library/asyncio.html>, 2026.
- Henry Qin, Qian Li, Jacqueline Speiser, Peter Kraft, and John Ousterhout. Arachne: Core-aware thread management. In *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 145–160, 2018.
- Redis Ltd. Redis: An in-memory data store. <https://redis.io/>, 2026.
- RedisAI. RedisAI: A redis module for serving tensors and executing deep learning models. <https://oss.redis.com/redisai/>, 2022.

- ScyllaDB. Seastar: A high-performance shared-nothing asynchronous C++ framework. <https://seastar.io/>, 2026.
- State Threads Library. State threads library for internet applications. <http://state-threads.sourceforge.net/>, 2009.
- The Go Authors. The Go programming language: Goroutines and the scheduler. <https://go.dev/>, 2026.
- Rob von Behren, Jeremy Condit, Feng Zhou, George C. Necula, and Eric Brewer. Capriccio: Scalable threads for internet services. In *Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP)*, pages 268–281, 2003.
- Matt Welsh, David Culler, and Eric Brewer. SEDA: An architecture for well-conditioned, scalable internet services. In *Proceedings of the 18th ACM Symposium on Operating Systems Principles (SOSP)*, pages 230–243, 2001.
- Irene Zhang, Amanda Raybuck, Pratyush Patel, Kirk Olynyk, Jacob Nelson, Omar S. Navarro Leija, Ashlie Martinez, Jing Liu, Anna Kornfeld Simpson, Sujay Jayakar, Pedro Henrique Penna, Max Demoulin, Piali Choudhury, and Anirudh Badam. The demikernel datapath OS architecture for microsecond-scale datacenter systems. In *Proceedings of the 28th ACM Symposium on Operating Systems Principles (SOSP)*, pages 195–211, 2021.